Code Review: WSAdmin (updated 2026-11-03)

Scope: whole program, about 17,600 lines of Swift (the earlier review covered only `validateSystem`; its findings are in `validateSystem Code Review.md` and are not repeated here). Method: read the recently changed files in full (`SheetVersionStore`, `TutorDetailLoadExtension`, `Student.unReassignTutor`, the `StudentMgmtVM` / `TutorMgmtVM` / `LocationMgmtVM` / `ServiceMgmtVM` diffs, `AppLogger`, `TokenStore`, `getFileID` / `readSheetCells` / `writeSheetCells`), scanned the whole tree for risky patterns, and built it (macOS, clean build: **succeeds, 140 warnings**). Nothing was run against live spreadsheets. Files not read line by line: `BillingVM`, `FinanceSummaryVM`, `RefDataVM`, most of `SystemVM` beyond the validation and timesheet-creation paths, and the SwiftUI views. Findings there come from compiler warnings and pattern scans only.

Clean results from the scans: no `try!`, `as!` or `fatalError`; tokens are no longer printed; no secrets in source; the four `*MgmtVM` files call the optimistic-lock preflight (4, 5, 11 and 11 call sites; I did not check that every write function has one).

Likely bugs

1. `deleteStudentSet` **overwrites earlier error text and logs nothing** (`StudentMgmtVM.swift`, the `catch` after `getFileID`, about line 609). It uses `errorMessage = ...` instead of `+=`, so every message collected for the previous students in the set is lost, and the failure is never sent to `AppLogger`.
2. **Data Counts are saved only if every student in the set succeeded** (`StudentMgmtVM.swift` ~line 537 and ~614). `decreaseActiveStudentCount()` runs in memory for each student that is marked Deleted, but `saveDataCounts()` is skipped when any other student in the set failed. The Student sheet then says N students are deleted while the persisted active count still includes them, until some later save happens to write the counts. The same shape (`if deleteResult { saveDataCounts }`) likely exists in `deleteTutorSet`; I did not confirm it. Save the counts whenever at least one record changed.
3. `Student.unReassignTutor` **leaves inconsistent tutor fields and has no guard** (`Student.swift` ~line 115).
   - When the first tutor is removed, `current` is copied from `previous`, but `previous` is left unchanged, so current and previous tutor are now the same tutor. It should become the tutor that was just removed (or be cleared).
   - If `tutorName` matches neither tutor, it still sets the status to Assigned and stamps the date.
   - Both log lines run after the status was already changed to Assigned, so they print "is Assigned. Reassigning...".
   - Logging uses detached `Task { }` blocks, so log order is not guaranteed.
4. `ensureTutorDetailsLoaded` **returns** `true` **for a Deleted tutor without loading anything** (`TutorDetailLoadExtension.swift` ~line 33). The doc comment says `false` means "load attempted and failed", and `tutorDetailsLoaded` stays `false`. Callers such as the unassign and billing paths then work on empty student and service arrays and report success. Either return a distinct result (e.g. an enum: loaded / skippedDeleted / failed) or have callers check the tutor status first.

5. **Results silently ignored** (compiler warnings):
    - `BillingVM.swift:777` ignores the result of `saveStudentData()` after billing updates the Student list.
    - `SystemVM.swift:1758` ignores `writeSheetCells` when writing the tutor name into a new Timesheet. That function returns `false` when there is no token.
    - `StudentMgmtVM.swift:234` `renameCurrentResult` is unused; the comment explains why a miss is fine, but a real failure (as opposed to "not found") is not logged.
6. `sheetDouble` **turns bad cells into** `0.0` **without a word** (`Utilities.swift:1573`). A mistyped cost or price in a spreadsheet becomes a $0 line in billing. Return an optional or log a warning with the cell value.
7. **Error-alert throttling is not race-safe** (`AppLogger.swift`, `sendErrorAlert`). `lastAlertSentAt` is set only after the network call, and `AppLogger` is an actor, so while one alert is awaiting the network a second error passes the same check and sends another email. Set the timestamp before the first `await` (and clear it on failure). Also note the alert body is the raw log message, which can contain student and tutor names, and it is emailed for every `.error`, including expected optimistic-lock conflicts.
8. `getFileID` **takes the first match silently, and its URL is built by string** (`Utilities.swift:18`). `getFolderID` warns when several folders match; `getFileID` does not, and a duplicate-named test file would be used without notice. It also builds the query with `addingPercentEncoding(.urlQueryAllowed)`, which leaves `&`, `+`, `?` and `=` unencoded, so a file name containing them is misread by the server. `getFolderID` already uses `URLComponents` correctly; use that. The same encoding is used for sheet ranges in `readSheetCells`, `writeSheetCells` and `SheetVersionStore.fetchValues`, where the range includes the tutor name.
9. `retryAfterDelay` **has a typo'd locale** (`Utilities.swift`, `Locale(identifier: " ")`). An HTTP-date `Retry-After` header probably fails to parse and falls back to the default backoff. Use `en_US_POSIX`. (The WSTutor copy already does.)
10. **Time bomb in the month pickers** (`BillingSelectionView.swift:20-21`, `ValidationMonthSelectionView.swift:18-19`). `yearArray[year - 2024]` and `.month!` crash with an index-out-of-range once the calendar year passes 2031 (the array ends there). Clamp the index.

Robustness and design

- **Multi-step changes are not atomic.** Assigning a student saves the Student list, then adds the row to the Tutor Details sheet, then saves the Tutor list; failing in the middle leaves the sheets disagreeing, and the only repair is to run validation by hand. The optimistic lock protects against outside edits, not against partial writes. Consider a distinct `PARTIAL FAILURE` log line naming what was and was not saved, and a "repair from validation results" path.
- **Lock cost and gaps.** `writeSheetCells` re-reads the range before every tracked write (about double the API calls), and there is a small window between that read and the write. The snapshot key is file plus start cell only, so two tracked reads that start at the same cell with different extents overwrite each other's snapshot. `verifyUnchanged` reads each snapshot one after another; multi-tutor operations do N sequential reads before starting.

- `AppLogger` **has its own retry loops** in `createDriveFile`, `updateDriveFile` and `sendErrorAlert` (504 only, one retry) instead of `performRequestWithRetry`. That is probably deliberate, since `performRequestWithRetry` logs an ERROR on failure and that would re-trigger an alert. Give `performRequestWithRetry` a `logFailures: false` option so the logger can share the retry logic (429 and 503 are not handled today).
- `runMode` **is a compile-time constant** (`WSAdmin.swift`, `let runMode: RunMode = .test`). Switching to production is a source edit and rebuild, which is easy to forget either way. Consider `#if DEBUG` or a scheme build setting, and show the mode in the window title.
- `unAssignService` (`ServiceMgmtVM.swift`): selecting several services in `ServiceListView` starts one `Task` per service, so they run concurrently, each re-checking the lock and saving tutor data. Run them in sequence. A service with zero tutors returns `false` with a WARNING that surfaces as an error alert; "nothing to do" is not a failure.
- `UserAuthVM`: `signIn()` returns silently when there is no main window, and a failed sign-in is only printed, never put in `errorMessage`. `getAuthScope` and `checkAuthScope` contain unused locals.

Consistency and cleanup

- **Log level:** 10 of 301 `ERROR:` messages are logged at the default `.info` level (9 in `StudentMgmtVM.swift`, 1 in `SystemVM.swift`), so they trigger no alert and are missed by level filters.
- **Messages returned to the UI:** `deleteLocationSet` joins `errorMessage += logMessage` with no newline, and `unDeleteLocationSet` appends INFO lines to the text shown for errors. Use one helper that appends a line.
- **Compiler warnings (140):** 63 unused immutable values, 15 unused initializations, 14 "defined but never used" (`if let x` that should be a boolean test), 13 unused variables, 8 written but never read. Worst files: `SystemVM` (23), `Utilities` (11), `StudentMgmtVM` (8), `BillingVM` (6). Notable ones: `default will never be executed` at `SystemVM.swift:488, 804, 910`; the optional printed with `\(emailAddress)` at `Utilities.swift:542`; `StudentMgmtVM.swift:502–503` (`result`, `studentBillingFileID` now unused after the shadowing refactor).
- **Duplicated constants:** `AppLogger` hard-codes the alert recipients that `PgmConstants` already holds (`russellEmail`, `writeSeattleEmail`).
- **Repeated three-line log pattern** (`print`, `AppLogger.shared.log`, append to message) appears several hundred times across the `*MgmtVM` files; one helper would remove most of it and make level and newline handling consistent.

Suggested order

1. Items 1, 2 and 5 (data loss or silent failure on save paths), and 3 (wrong tutor fields after unreassign).
2. Items 4 and 6 (silent success or silent zero), then 7 and 8.
3. Items 9 and 10 (small), then the warning cleanup in one pass.